CyberDefenders

RESOLUCIÓN DE LABORATORIO

Lab: RE101 (CyberDefenders)

Análisis de diversos artefactos mediante técnicas de ingeniería inversa, incluyendo binarios, scripts ofuscados y archivos corruptos, para la identificación de datos ocultos y comprensión de mecanismos de codificación, cifrado y ofuscación.

Sebastian David
Torres Reyes

Escenario:

El desafío RE101 es un ejercicio de análisis binario, una tarea que realizan los analistas del equipo azul de seguridad para comprender cómo funciona un malware específico y extraer información de inteligencia.

Herramientas utilizadas:

- IDA
- CyberChef
- zipdetails
- strings
- Hexed.it

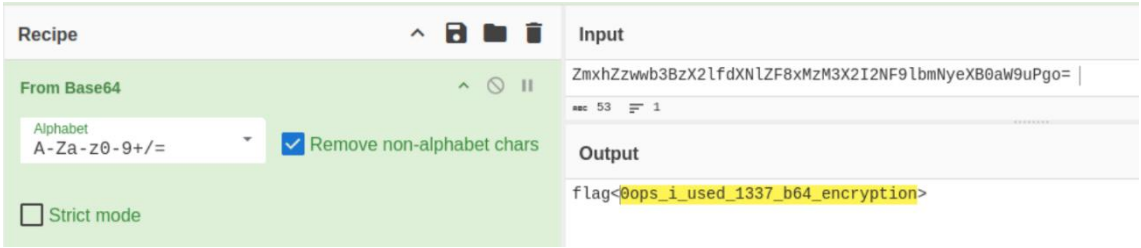
P1: Archivo: MALWARE000 - He usado este nueva encriptación de la que oí hablar en línea para mi software pirata; ¡apuesto a que no puedes extraer la bandera!

R: 0ops_i_used_1337_b64_encryption

Mediante el visor hexadecimal de IDA se visualizó que el archivo MALWARE000 está en formato ELF (Executable and Linkable Format), un formato estándar para archivos ejecutables de Linux.

Se encontró una cadena de texto que usa padding (=), un indicador de que está codificado en Base64. Se utilizó la herramienta CyberChef para la decodificación. El flag es “flag<0ops_i_used_1337_b64_encryption>”

```
0000000000400D40  41 5E 41 5F C3 90 66 2E  0F 1F 84 00 00 00 00 00  A^A_Ð·f.........
0000000000400D50  F3 C3 00 00 48 83 EC 08  48 83 C4 08 C3 00 00 00  ....H...........
0000000000400D60  01 00 02 00 5A 6D 78 68  5A 7A 77 77 62 33 42 7A  ....ZmxhZzwwb3Bz
0000000000400D70  58 32 6C 66 64 58 4E 6C  5A 46 38 78 4D 7A 4D 33  X2lfdXNlZF8xMzM3
0000000000400D80  58 32 49 32 4E 46 39 6C  62 6D 4E 79 65 58 42 30  X2I2NF9lbmNyeXB0
0000000000400D90  61 57 39 75 50 67 6F 3D  00 49 66 20 79 6F 75 20  aW9uPgo=.If·you·
0000000000400DA0  73 65 65 20 74 68 69 73  20 6D 65 73 73 61 67 65  see·this·message
0000000000400DB0  2E 2E 2E 0A 00 2F 64 65  76 2F 75 72 61 6E 64 6F  ...../dev/urando
0000000000400DC0  6D 00 72 62 00 07 25 73  00 0A 3E 00 20 00 59 00  m.rb..%s..>.·.Y.
0000000000400DD0  30 00 55 00 48 00 34 00  56 00 33 00 42 00 45 00  0.U.H.4.V.3.B.E.
0000000000400DE0  4E 00 57 00 6E 00 44 00  21 00 31 00 21 0A 00 00  N.W.n.D.!.1.!...
0000000000400DF0  01 1B 03 3B 3C 00 00 00  06 00 00 00 70 F7 FF FF  ...;<.......p...
```



P2: Archivo: Just some JS - ¡Mira lo que puedo hacer!

R: what_a_cheeky_language!1!

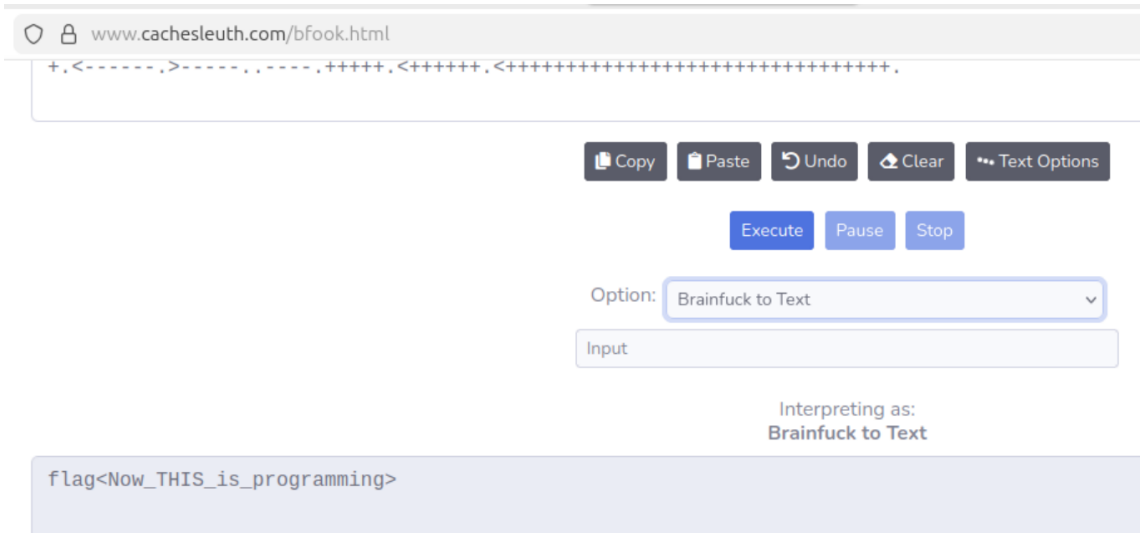
El lenguaje de programación utilizado es JS y el método empleado es JSFuck. Esta técnica de programación esotérica permite escribir cualquier código de JS utilizando solamente 6 caracteres: [,], (,), ! y +. Se utilizó una herramienta de decodificación JSFuck para obtener el código en su versión estándar. El código JS es console.log(“flag<what_a_cheeky_language!1!>”).

P3: Archivo: This is not JS. Estoy cansado de Javascript. ¡Por suerte, encontré al abuelo de ese lenguaje tan aburrido!

R: Now_THIS_is_programming

El lenguaje de programación utilizado se llama Brainfuck debido al uso de los caracteres tales como: +, ., <, >, etc.

Se utilizó un Brainfuck Interpreter para visualizar el código en texto legible. Como resultado se obtiene la cadena de texto “flag<Now_THIS_is_programming>”



P4: Archivo: Unzip Me - Comprimí flag.txt y lo cifré con la contraseña "password", pero creo que el encabezado se dañó... Puedes obtener la bandera si arreglas el archivo.

R: R3ad_th3_spec

Mediante la herramienta zipdetails se mostró que el archivo está truncado (cabecera corrupta) revelando que solamente se encuentran 185 bytes de 22616 bytes.

```
0000 LOCAL HEADER #1       04034B50
0004 Extract Zip Spec      0A '1.0'
0005 Extract OS            00 'MS-DOS'
0006 General Purpose Flag  0009
     [Bit  0]              1 'Encryption'
     [Bit  3]              1 'Streamed'
0008 Compression Method    0000 'Stored'
000A Last Mod Time         508EAB5A 'Tue Apr 14 21:26:52 2020'
000E CRC                   95672256
0012 Compressed Length     00000020
0016 Uncompressed Length   00000014
001A Filename Length       5858
001C Extra Length          001C
Truncated file (got 185, wanted 22616):
```

Adicionalmente se utilizó strings y se encontró que el archivo comprimido tiene de nombre flag.txt, es decir, un Filename Length de 8 bytes.

```
e.zip_broken
PV"g
flag.txtUT
PV"g
flag.txtUT
```

Además, zipdetails muestra que el valor de Filename Length es 5858; el estándar ZIP usa el sistema Little Endian por lo que lee los bytes al revés, es decir, el valor hexadecimal 58 58 del Filename Length traducido a decimal es 22616 bytes lo cual no es correcto ya que en realidad es de 8 bytes; aquí es donde se muestra la manipulación de cabecera (Header Tampering). Se utilizó Hexed.it para corregir los valores hexadecimales erróneos, el offset del Filename Length en los archivos ZIP es 0x1A, donde se encuentra el valor 58 58. Se sobre escribieron los valores hexadecimales por 08 00 (8 bytes en formato decimal) y se exportó el archivo.

```
-Untitled- ×   file.zip_broken ×
00000000  50 4B 03 04 0A 00 09 00 00 00 5A AB 8E 50 56 22  PK........Z½ÄPV"
00000010  67 95 20 00 00 00 14 00 00 00 08 00 1C 00 66 6C  gò .......¸....fl
00000020  61 67 2E 74 78 74 55 54 09 00 03 DC 62 96 5E DC  ag.txtUT...▄bû^▄
00000030  62 96 5E 75 78 0B 00 01 04 E8 03 00 00 04 E8 03  bû^ux....Φ....Φ.
00000040  00 00 71 A9 F7 10 B7 9C 45 63 4A ED 2E DC C9 D6  ..q⌐≈.╗£EcJφ.▄╔╓
00000050  ED BD E4 B5 99 69 C5 AB 94 FF 64 22 DE FF 70 78  φ╝Σ╡Öi┼½ö d"▌ px
00000060  8C E7 50 4B 07 08 56 22 67 95 20 00 00 00 14 00  îτPK..V"gò .....
00000070  00 00 50 4B 01 02 1E 03 0A 00 09 00 00 00 5A AB  ..PK..........Z½
00000080  8E 50 56 22 67 95 20 00 00 00 14 00 00 00 08 00  ÄPV"gò .........
00000090  18 00 00 00 00 00 01 00 00 00 A4 81 00 00 00 00  ..........ñü....
000000A0  66 6C 61 67 2E 74 78 74 55 54 05 00 03 DC 62 96  flag.txtUT...▄bû
000000B0  5E 75 78 0B 00 01 04 E8 03 00 00 04 E8 03 00 00  ^ux....Φ....Φ...
000000C0  50 4B 05 06 00 00 00 00 01 00 01 00 4E 00 00 00  PK..........N...
000000D0  72 00 00 00 00 00 0A                             r......
```

Se logró descomprimir el archivo ZIP utilizando la contraseña “password”. El contenido del archivo flag.txt es “flag<R3ad_th3_spec>”

P5: Archivo: MALWARE101 - Al parecer, mi cifrado no es tan seguro. ¡Tengo una nueva forma de ocultar mis banderas!

R: sTaCk_strings_LMAO

Al visualizar los hexadecimales del malware mediante IDA se descubrieron cadenas ofuscadas.

```
IDA View-A   Hex View-1   Local Ty
00000000004004B0  F3 C3 0F 1F 40 00 66 2E 0F 1F 84 00 00 00 00 00  ....@.f.........
00000000004004C0  55 48 89 E5 5D EB 89 66 0F 1F 84 00 00 00 00 00  UH.....f........
00000000004004D0  55 48 89 E5 48 81 EC C0 01 00 00 48 BF 44 06 40  UH.........H.D.@
00000000004004E0  00 00 00 00 00 C7 45 FC 00 00 00 00 C6 85 60 FE  ............b·`.
00000000004004F0  FF FF 67 C6 85 57 FE FF FF 61 C6 85 5D FE FF FF  ..gb·W...ab·]...
0000000000400500  72 C6 85 5E FE FF FF 69 C6 85 5F FE FF FF 6E C6  rb·^...ib·_...n.
0000000000400510  85 55 FE FF FF 73 C6 85 67 FE FF FF 3E C6 85 68  .U...sb·g...>b·h
0000000000400520  FE FF FF 00 C6 85 59 FE FF FF 6B C6 85 61 FE FF  ....b·Y...kb·a..
0000000000400530  FF 73 C6 85 52 FE FF FF 61 C6 85 56 FE FF FF 54  .sb·R...ab·V...T
0000000000400540  C6 85 5A FE FF FF 5F C6 85 62 FE FF FF 5F C6 85  b·Z..._b·b..._b·
0000000000400550  51 FE FF FF 6C C6 85 50 FE FF FF 66 C6 85 5B FE  Q...lb·P...fb·[.
0000000000400560  FF FF 73 C6 85 5C FE FF FF 74 C6 85 63 FE FF FF  ..sb·\...tb·c...
0000000000400570  4C C6 85 58 FE FF FF 43 C6 85 65 FE FF FF 41 C6  Lb·X...Cb·e...A.
0000000000400580  85 66 FE FF FF 4F C6 85 53 FE FF FF 67 C6 85 54  .f...Ob·S...gb·T
0000000000400590  FE FF FF 3C C6 85 64 FE FF FF 4D B0 00 E8 2E FE  ...<b·d...M.....
00000000004005A0  FF FF 31 C9 89 85 4C FE FF FF 89 C8 48 81 C4 C0  ..1ɟ·.L.........
00000000004005B0  01 00 00 5D C3 66 2E 0F 1F 84 00 00 00 00 00 90  ...]............
00000000004005C0  41 57 41 56 49 89 D7 41 55 41 54 4C 8D 25 3E 08  AWAVI...UATL.%>.
00000000004005D0  20 00 55 48 8D 2D 3E 08 20 00 53 41 89 FD 49 89  ·.UH.->.·.SA..I.
00000000004005E0  F6 4C 29 E5 48 83 EC 08 48 C1 FD 03 E8 B7 FD FF  ....H...........
00000000004005F0  FF 48 85 ED 74 20 31 DB 0F 1F 84 00 00 00 00 00  .H....1.........
0000000000400600  4C 89 FA 4C 89 F6 44 89 EF 41 FF 14 DC 48 83 C3  L..L.....A......
0000000000400610  01 48 39 DD 75 EA 48 83 C4 08 5B 5D 41 5C 41 5D  .H9.......[]A\A]
0000000000400620  41 5E 41 5F C3 90 66 2E 0F 1F 84 00 00 00 00 00  A^A_Ð·f.........
0000000000400630  F3 C3 00 00 48 83 EC 08 48 83 C4 08 C3 00 00 00  ....H...........
0000000000400640  01 00 02 00 4E 6F 74 68 69 6E 67 20 74 6F 20 73  ....Nothing·to·s
0000000000400650  65 65 20 68 65 72 65 2E 2E 2E 2E 0A 00 00 00 00  ee·here.........
0000000000400660  01 1B 03 3B 3C 00 00 00 06 00 00 00 60 FD FF FF  ...;<.......`...
0000000000400670  98 00 00 00 80 FD FF FF 58 00 00 00 B0 FD FF FF  ........X.......
```

Al analizar el main del código fuente del malware se revela que la técnica de ofuscación empleada es “Stack Strings”.

```
; Attributes: bp-based frame

; int main(int, char **, char **)
main proc near

var_1B4= dword ptr -1B4h
var_1B0= byte ptr -1B0h
var_1AF= byte ptr -1AFh
var_1AE= byte ptr -1AEh
var_1AD= byte ptr -1ADh
var_1AC= byte ptr -1ACh
var_1AB= byte ptr -1ABh
var_1AA= byte ptr -1AAh
var_1A9= byte ptr -1A9h
var_1A8= byte ptr -1A8h
var_1A7= byte ptr -1A7h
var_1A6= byte ptr -1A6h
var_1A5= byte ptr -1A5h
var_1A4= byte ptr -1A4h
var_1A3= byte ptr -1A3h
var_1A2= byte ptr -1A2h
var_1A1= byte ptr -1A1h
var_1A0= byte ptr -1A0h
var_19F= byte ptr -19Fh
var_19E= byte ptr -19Eh
var_19D= byte ptr -19Dh
var_19C= byte ptr -19Ch
var_19B= byte ptr -19Bh
var_19A= byte ptr -19Ah
var_199= byte ptr -199h
var_198= byte ptr -198h
var_4= dword ptr -4

; __unwind {
push    rbp
mov     rbp, rsp
sub     rsp, 1C0h
mov     rdi, offset format ; "Nothing to see here....\n"
mov     [rbp+var_4], 0
mov     [rbp+var_1A0], 67h ; 'g'
mov     [rbp+var_1A9], 61h ; 'a'
mov     [rbp+var_1A3], 72h ; 'r'
mov     [rbp+var_1A2], 69h ; 'i'
mov     [rbp+var_1A1], 6Eh ; 'n'
mov     [rbp+var_1AB], 73h ; 's'
mov     [rbp+var_199], 3Eh ; '>'
mov     [rbp+var_198], 0
mov     [rbp+var_1A7], 6Bh ; 'k'
mov     [rbp+var_19F], 73h ; 's'
mov     [rbp+var_1AE], 61h ; 'a'
mov     [rbp+var_1AA], 54h ; 'T'
mov     [rbp+var_1A6], 5Fh ; '_'
mov     [rbp+var_19E], 5Fh ; '_'
mov     [rbp+var_1AF], 6Ch ; 'l'
mov     [rbp+var_1B0], 66h ; 'f'
mov     [rbp+var_1A5], 73h ; 's'
mov     [rbp+var_1A4], 74h ; 't'
mov     [rbp+var_19D], 4Ch ; 'L'
mov     [rbp+var_1A8], 43h ; 'C'
mov     [rbp+var_19B], 41h ; 'A'
mov     [rbp+var_19A], 4Fh ; 'O'
mov     [rbp+var_1AD], 67h ; 'g'
mov     [rbp+var_1AC], 3Ch ; '<'
mov     [rbp+var_19C], 4Dh ; 'M'
mov     al, 0
call    _printf
xor     ecx, ecx
mov     [rbp+var_1B4], eax
mov     eax, ecx
add     rsp, 1C0h
pop     rbp
retn
; } // starts at 4004D0
main endp
```

Se ordena por offset de menor a mayor dirección. Como resultado se obtiene la cadena “flag< sTaCk_strings_LMAO>”

P6: Archivo: MALWARE201 - Uf... Supongo que tendré que crear mi propio cifrado. No soy muy bueno en matemáticas, ¡pero me parece bien!

R: malwar3-3ncryp710n-15-Sh17

Mediante el análisis del MALWARE201 en IDA se determinó que se utilizó el método de esquema de cifrado tipo flujo basado en XOR de manera dinámica al depender de una variable “i”; se localizó unk_40082B para la extracción de los valores hexadecimales cifrados.

```
.rodata:0000000000400824 format          db '\x%02x',0          ; DATA XREF: sub_4005B0+26↑o
.rodata:000000000040082B unk_40082B      db  6Dh ; m            ; DATA XREF: main+12↑o
.rodata:000000000040082C                 db  78h ; x
.rodata:000000000040082D                 db  61h ; a
.rodata:000000000040082E                 db  6Ch ; l
.rodata:000000000040082F                 db 0DDh
.rodata:0000000000400830                 db  7Eh ; ~
.rodata:0000000000400831                 db  65h ; e
.rodata:0000000000400832                 db  7Eh ; ~
.rodata:0000000000400833                 db  47h ; G
.rodata:0000000000400834                 db  6Ah ; j
.rodata:0000000000400835                 db  4Fh ; O
.rodata:0000000000400836                 db 0CCh
.rodata:0000000000400837                 db 0F7h
.rodata:0000000000400838                 db 0CAh
.rodata:0000000000400839                 db  73h ; s
.rodata:000000000040083A                 db  68h ; h
.rodata:000000000040083B                 db  55h ; U
.rodata:000000000040083C                 db  42h ; B
.rodata:000000000040083D                 db  53h ; S
.rodata:000000000040083E                 db 0DCh
.rodata:000000000040083F                 db 0D7h
.rodata:0000000000400840                 db 0D4h
.rodata:0000000000400841                 db  6Bh ; k
.rodata:0000000000400842                 db 0ECh
.rodata:0000000000400843                 db 0DBh
.rodata:0000000000400844                 db 0D2h
.rodata:0000000000400845                 db 0E1h
.rodata:0000000000400846                 db  1Ch
.rodata:0000000000400847                 db  6Dh ; m
.rodata:0000000000400848                 db 0DEh
.rodata:0000000000400849                 db 0D1h
.rodata:000000000040084A                 db 0C2h
.rodata:000000000040084B                 db    0
.rodata:000000000040084C ; const char aTheEncryptedFl[]
.rodata:000000000040084C aTheEncryptedFl db 'The encrypted flag is: "',0
.rodata:000000000040084C                                        ; DATA XREF: main+8↑o
.rodata:0000000000400865 ; const char asc_400865[]
.rodata:0000000000400865 asc_400865      db '"',0Ah             ; DATA XREF: main+3A↑o
.rodata:0000000000400867                 db 0Ah,0
.rodata:0000000000400869 aMyMessage      db 'my message',0      ; DATA XREF: main+55↑o
.rodata:0000000000400874 aEncryptSLd     db 'encrypt("%s", %ld) == "',0
.rodata:0000000000400874                                        ; DATA XREF: main+4B↑o
.rodata:000000000040088C ; const char asc_40088C[]
.rodata:000000000040088C asc_40088C      db '"',0Ah,0           ; DATA XREF: main+AE↑o
.rodata:000000000040088C _rodata         ends
.rodata:000000000040088C
```

Al ser una operación dinámica se utilizó un script de Python, el flujo de descifrado inicia tomando un byte de la lista de los valores hexadecimales, luego se calcula la clave única para esa posición. La variable i cambia en cada paso. Después, se aplica el XOR entre el byte cifrado y la clave generada. Finalmente, se realiza el desplazamiento de los bits una posición a la derecha. Como resultado se obtiene la cadena de texto “flag<malwar3-3ncryp710n-15-Sh17>”

```python
data = [0x6d, 0x78, 0x61, 0x6c, 0xdd, 0x7e, 0x65, 0x7e, 0x47, 0x6a, 0x4f, 0xcc, 0xf7, 0xca, 0x73, 0x68,


flag = ""
for i in range(len(data)):
    # 1. Calcular la clave dinámica: (i % 0xFF) | 0xA0
    key = (i % 0xFF) | 0xA0

    # 2. Operación XOR
    xored = data[i] ^ key

    # 3. Desplazamiento lógico a la derecha de 1 (>>> 1)
    decoded_byte = (xored >> 1) & 0xFF

    flag += chr(decoded_byte)

print(f"La bandera es: {flag}")
```

```
La bandera es: flag<malwar3-3ncryp710n-15-Sh17>
```